An Empirical Study on the Efficacy of ESLint in Reducing Bugs in TypeScript

Project Proposal - CMPT 479/982 - Fall 2025 - Professor Saba Alimadadi

Sara Magdalinski (sma274@sfu.ca) & Nicholas Tait (nta52@sfu.ca)

I. MOTIVATION AND PROBLEM STATEMENT

Static analysis tools, such as linters, are widely used in modern software development to identify potential coding issues, enforce best practices, and improve code readability. In the JavaScript and TypeScript ecosystems, ESLint has emerged as the most popular linter [1], [2]. However, despite its popularity, there is limited quantitative evidence regarding whether using a linter actually leads to fewer bugs in real-world projects.

Existing research primarily investigates why and how developers use linters, including which rules they enable and why they disable certain warnings, but it does not evaluate the effectiveness of linting in reducing software defects [1] - [3]. Other studies which extend to Java, Ruby, and Python ecosystems have examined static analysis usage patterns, but often stop short of connecting tool adoption to observable quality outcomes like bug-fix frequency [4], [5].

Our project aims to fill this gap by empirically studying the relationship between linting practices and bug-fix activity across real-world TypeScript repositories hosted on GitHub. Specifically, we will investigate whether projects that use ESLint tend to have a lower proportion of bug-fix commits and whether the strictness of linting configurations correlates with software quality indicators.

A. Related Work & Contribution

Prior work in other ecosystems has examined the effectiveness of static analysis on real defects. [6] measured false negatives of FindBugs/PMD/JLint on Java projects by running tools on pre-fix snapshots and aligning warnings with faulty lines, and found that many field defects were not flagged, and effectiveness varied by warning family. We adapt this snapshot-and-alignment method to ESLint to estimate the share of linter-preventable fixes in TypeScript. Unlike prior work, we also analyze project-level outcomes (bug-fix ratio) and the role of linter adoption/strictness, controlling for popularity and age.

[7] studied PMD warning density across 23 Apache projects and asked whether bug-inducing files/changes carry more warnings than others. They found statistically significant but negligible effects overall, with slightly stronger signals when restricting to a curated default rule set and using a just-in-time comparison.

Building on these insights, our study reports project-level outcomes and linter strictness, and includes a commit-level deep dive of linter-preventable fixes on pre-fix snapshots. We also account for time trends in warning density and choose a curated ESLint rule subset in sensitivity analyses.

II. RESEARCH QUESTIONS

RQ1: How prevalent is the adoption of ESLint among open-source TypeScript Projects, and how strict are their configurations?

RQ2: Do TypeScript projects that use ESLint exhibit lower bug-fix ratios compared to those that do not?

RQ3: Is there a relationship between the strictness of a project's ESLint configuration and its bug-fix activity?

RQ4: For a subset of bug-fix commits, what fraction are linter-preventable under (a) the project's actual config (if available) and (b) a standard TypeScript ESLint config?

III. PROPOSED APPROACH AND METHODOLOGY

A. Data Collection

We will use the GitHub REST API to extract several non-forked, active TypeScript repositories containing ESLint and those containing no linters. For each repository, we will extract metadata, including the creation date and the commit history.

B. Linter Detection and Strictness:

To identify the presence of ESLint within the repositories, we will check for the presence of ESLint configuration files (e.g eslint.config.js) in the root directory. This way, we will be able to parse this file to extract the defined rules and perform analysis on these rules with respect to bug detection and prevention. We will also measure the strictness of the configurations in two ways:

Absolute Strictness: The total number of rules configured as "error".

Relative Strictness: The ratio of "error" rules to the total number of enabled rules.

This analysis will enable us to assess how strictly projects enforce ESLint rules and whether stricter configurations correlate with lower bug-fix rates.

C. Bug-Fix Identification:

We will identify bug-fix commits by parsing commit messages for bug-related keywords and cross-checking linked issue labels where available. The bug fix ratio will be computed as the number of bug-fix commits divided by the total number of commits.

D. Linter-Preventable Deep Dive:

For a subset of bug-fix commits, we will follow the approach of [6] by downloading the pre-fix version of each file and running ESLint on that snapshot. We will evaluate two conditions:

- Using the project's actual ESLint configuration (if available).
- Using a standardized ESLint configuration.

A fix will be labelled as linter-preventable if any ESLint finding from either configuration corresponds to a changed line in the bug-fix diff. This analysis will quantify how many real-world defects could have been detected earlier by ESLint.

E. Data Analysis

1) *Research Question 1:* For this question, we plan to calculate the proportion of TypeScript repositories in our dataset that have a detectable ESLint configuration. This will be presented as a simple percentage. For repositories using ESLint, we will quantify strictness by calculating a strictness score. We will do this in two ways, as mentioned in the section "linter detection and strictness" above.

2) *Research question 2:* We plan to compare projects that contain ESLint to those that do not. Based on the Shapiro-Wilk test results, we will perform the corresponding analysis depending on whether the data is normally distributed or not. If the data is normally distributed, we will use an independent sample t-test to compare the mean bug-fix ratios of the two groups. If the data is not normally distributed, we will use the Mann-Whitney U test. In addition to checking for statistical significance, we will calculate the effect size to quantify the magnitude of the difference between the two groups.

3) Research Question 3: This question investigates the relationship between strictness and bug-fix ratio. We plan to calculate a correlation coefficient using an appropriate method based on whether the data is normally distributed or not. This preliminary analysis will provide insight into the relationship between the strictness of the configuration file and the bug-fix ratio. To further analyze this relationship, we plan to also build a multiple linear regression model that takes into account the project age, project size, activity level, as well as potentially more factors, to determine whether there is still a correlation between stricter linting and bug fix ratios even after considering external factors.

4) Research Question 4: This analysis is primarily descriptive and will focus on a small subset of manually analyzed bug-fix commits. We plan to calculate the percentage of bug-fixes that are deemed "linter-preventable" under two conditions, as mentioned above. Then we will use a McNemar's test to determine if there is a statistically significant difference between the proportion of bugs caught by the project's ESLint configuration versus the standard ESLint configuration. We also plan to categorize the types of preventable bugs identified. This will be done by manually analyzing the warnings generated by ESLint and determining whether the warning is highly likely to have caught the bug, moderately likely to have caught the bug, or not likely to have caught the bug.

IV. EXPECTED OUTCOMES

This study will produce both empirical evidence and reusable research artifacts for the software engineering community. Specifically, we expect to deliver:

1. A curated dataset of open-source TypeScript repositories in CSV format, which will be made publicly available to support replication and follow-up research, annotated with:

- Linter Adoption indicators (presence/absence of ESLint and configuration files).
- Quantitative measures of linter strictness.
- Bug-fix activity metrics (e.g., bug-fix ratios and commit-level classifications).

2. Empirical evidence on whether linter adoption is associated with improved software quality, measured through lower bug-fix ratios. We will also examine whether projects with stricter linter configurations demonstrate stronger correlations with these quality outcomes.

3. A commit-level analysis estimating the fraction of bug-fix commits that are linter-preventable, comparing (a) each project's actual ESLint configuration (if used) and (b) a standardized TypeScript ESLint configuration. This analysis will identify the most frequently implicated ESLint rules and measure the potential benefits of stricter configurations. It will also attest to the soundness of our bug-fix ratio calculation by verifying that a

representative portion of commits labelled as “bug fixes” correspond to true code defects that could have been prevented through static analysis.

4. A comprehensive written report summarizing the study’s design, data, analysis, and findings. The report will discuss key insights, limitations, and implications, and will be accompanied by visualizations to present the results clearly.

V. TIMELINE

Oct 20: Submit proposal.

Oct 20-22: Complete literature review of empirical studies on static analysis and linting.

Oct 23-Nov 5: Data Collection and cleaning. Identify ESLint configurations and compute strictness metrics.

Nov 6-Nov 14: Bug-fix detection and linter-preventable deep-dive analysis.

Nov 15-Nov 20: Statistical analysis and presentation preparation.

Nov 21-Dec 2: Finalize paper, figures, and dataset for submission.

REFERENCES

[1] K. F. Tómasdóttir, M. Aniche, and A. Van Deursen, “The Adoption of JavaScript Linters in Practice: A Case Study on ESLint,” IEEE Transactions on Software Engineering, vol. 46, no. 8, pp. 863–891, Aug. 2020, doi: 10.1109/TSE.2018.2871058.

[2] K. F. Tómasdóttir, M. Aniche, and A. van Deursen, “Why and how JavaScript developers use linters,” in 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), Oct. 2017, pp. 578–589. doi: 10.1109/ASE.2017.8115668.

[3] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, “Why don’t software developers use static analysis tools to find bugs?,” in 2013 35th International Conference on Software Engineering (ICSE), May 2013, pp. 672–681. doi: 10.1109/ICSE.2013.6606613.

[4] M. Beller, R. Bholanath, S. McIntosh, and A. Zaidman, “Analyzing the State of Static Analysis: A Large-Scale Evaluation in Open Source Software,” in 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER), Mar. 2016, pp. 470–481. doi: 10.1109/SANER.2016.105.

[5] C. Vassallo, S. Panichella, F. Palomba, S. Proksch, H. C. Gall, and A. Zaidman, “How developers engage with static analysis tools in different contexts,” Empir Software Eng, vol. 25, no. 2, pp. 1419–1457, Mar. 2020, doi: 10.1007/s10664-019-09750-5.

[6] F. Thung, Lucia, D. Lo, L. Jiang, F. Rahman, and P. T. Devanbu, “To what extent could we detect field defects? an empirical study of false negatives in static bug finding tools,” in Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering, Essen Germany: ACM, Sept. 2012, pp. 50–59. doi: 10.1145/2351676.2351685.

[7] A. Trautsch, S. Herbold, and J. Grabowski, “Are automated static analysis tools worth it? An investigation into relative warning density and external software quality on the example of Apache open source projects,” Empir Software Eng, vol. 28, no. 3, p. 66, Apr. 2023, doi: 10.1007/s10664-023-10301-2.